

PROJECT	Alpha Pride Security — Black Hawk SOC-OS
PURPOSE	Enterprise Security Operations Platform for a Kenyan private security company
REPOSITORY	github.com/Morpheus-cain/Alpha-Pride-Security
FORKED FROM	jibrilrashyd-sketch/Black-Hawk
SCAFFOLDED WITH	Lovable (AI-assisted frontend)
CONSULTANT	Immersicloud Consulting · immersitec@gmail.com

1 · TECH STACK

LAYER	TECHNOLOGY	LAYER	TECHNOLOGY
Framework	React 18 + TypeScript + Vite 5	Database	Supabase (Postgres + RLS)
Styling	Tailwind CSS + shadcn/ui (Radix)	Auth	Supabase Auth (JWT + Roles)
Routing	React Router v6	Edge Functions	Deno TypeScript + OpenAI
Server State	TanStack Query v5	Realtime	Supabase Realtime (WS)
Animation	Framer Motion	Storage	Supabase Storage (S3)
Maps	Mapbox GL	AI/ML	OpenAI GPT-4o via Edge Fns
PDF Export	jsPDF + jspdf-autotable	Client	src/integrations/supabase/client.ts
Forms	React Hook Form + Zod		src/integrations/supabase/types.ts
Charts	Recharts	Lazy Load	safeLazy() — NEVER React.lazy()

Key import rules: Always import supabase from "@/integrations/supabase/client" · Always wrap pages with safeLazy() not React.lazy · Never read roles from sessionStorage.

2 · AUTH & PERMISSIONS SYSTEM

How Auth Works

#	STEP
1	AuthProvider in useAuth.ts wraps the entire app
2	Auth state set ONLY from onAuthStateChange — never getSession()
3	On login, role is fetched from user_roles table via fetchUserRole(userId)
4	Role stored in AuthContext.userRole — server-side source of truth
5	signOut uses scope:'global' to revoke all server sessions

Permission Levels (Escalating)

LEVEL	CAPABILITY	USE CASE
none	Hidden, route blocked, API rejected	No access at all
view	Read-only access	Dashboard observers
edit	Read + update existing records	Officers, operators
create	Read + update + create new (implies edit)	Supervisors, managers
delete	Full CRUD including destructive ops	Executives, system admin

Role → Module Access Matrix

ROLE	KEY MODULES & PERMISSIONS
ceo	Executive dashboard (full CRUD), all others view-only
coo	Full ops: control room, incidents, alarms, fleet, CIT, patrol, CCTV, MDT
gm	Client management (full), contracts, finance view, incident view
ops_manager	Full ops: dispatch, incidents, alarms, MDT, patrol, scheduling
control	Control room (full), incidents (create), alarms (edit), CCTV/bodycam (view)
hr / hr_officer	Full HR suite: staff, leave, training, documents, scheduling
finance / finance_officer	Full finance: billing, invoicing, bank rec, expenses, loss control
payroll_officer	Pay runs, payslips, statutory (PAYE/NHIF/NSSF/NITA)
cit_manager / cit_officer	CIT runs, manifests, vault, routes, crews, incidents
courier_manager	Full courier: deliveries, dispatch, riders, routes, incidents
system_admin	Users, roles, settings, tenants — NO operational data
compliance	Full governance: compliance register, audit, policies, approvals
customer_service_manager	Full CS: tickets, complaints, team, config

3 · DATABASE SCHEMA

Applied Tables (~75 total in supabase/types.ts)

CATEGORY	TABLES
Authentication	profiles, user_roles
Staff / HR	staff, schedules, attendance, leave_records, staff_certifications
Sites / Clients	sites, clients, client_contacts, client_finances, contracts
Incidents	incidents, incident_evidence, mobile_incidents
Patrols	patrols, patrol_checkpoints, mobile_patrols, welfare_events
Alarms	alarm_activations, alarm_sensors, sos_alerts
Dispatch / MDT	dispatch_logs, dispatch_requests, mdt_messages, operator_statuses
Communications	calls, comms_records, sms_messages, whatsapp_messages, communication_tickets
Body Cam / Evidence	body_cam_clips, body_cam_devices, body_cam_footage, clip_chain_of_custody
Loss Control	loss_control_records, loss_control_behavior_patterns, loss_control_corrective_actions
Technical Security	technical_client_sites, technical_equipment, technical_work_orders, technical_maintenance_schedules
Fleet / Events	vehicles, event_staff_assignments, security_events
Training	training_programs, training_records, training_sessions
Clients / Contracts	documents, document_access_logs, reports, risk_assessments
Advisory / Intel	strategic_advisories, strategic_advisory_audit, advisory_history
Governance	audit_trail, tasks, client_complaints, cs_categories

DB Conventions

PATTERN	RULE
Primary keys	id UUID PRIMARY KEY DEFAULT gen_random_uuid()

PATTERN	RULE
Timestamps	created_at / updated_at TIMESTAMPTZ DEFAULT NOW()
Auth FK	user_id UUID REFERENCES public.profiles(id) – never auth.users directly
Role check	public.has_role(auth.uid(), 'ceo':app_role)
Elevated check	public.is_elevated_user(auth.uid()) – ceo, coo, ops_supervisor, system_admin
RLS	ALTER TABLE public.X ENABLE ROW LEVEL SECURITY; on every new table

Pending Migrations (in .lovable/pending_migrations/ — NOT YET APPLIED)

FILE	PURPOSE	PRIORITY
hr_finance_payroll_governance.sql	Payroll runs, payslips, expense claims, disciplinary, performance reviews	Phase D
cit_module.sql	CIT runs, manifests, vault, routes tables	Phase C
k9_module.sql	K9 dogs, handlers, deployments, incidents	Phase C
visitor_access_armoury.sql	Visitor pre-clearance, badge log, armoury custody chain	Phase C
cmc_module.sql	Crisis Management Centre activations, decisions, resources	Phase C
cctv_cameras.sql	Camera registry, zones, recording schedule, status	Phase C
field_portal_phase2.sql	Patrol sessions, checkpoint scans, patrol gaps view	Phase B
hq_connect.sql	HQ directives, branch acknowledgements, directive log	Phase B
auto_dispatch_rules.sql	Auto-dispatch rule engine (Part of Phase 7)	Phase B
dob_soft_delete.sql	Soft delete for DOB entries (deleted_at, deleted_by)	Phase B
incident_command_v2.sql	Incident command structure, ICS task assignments	Phase C
enterprise_first_wave.sql	Multi-branch / entity tenant structures	Phase H
directives_audit.sql	Directive log and acknowledgement tracking	Phase F

Phase 6 and Phase 7 SQL is documented in PHASE6_MIGRATION.md and PHASE7_MIGRATION.md — these have been written but must be applied via Lovable Cloud → Database → SQL Editor.

4 · EDGE FUNCTIONS

All edge functions live in supabase/functions/. Written in Deno TypeScript. Require OPENAI_API_KEY secret set in Supabase. Deploy via: supabase functions deploy <name>

FUNCTION	LIN ES	PURPOSE	FUNCTION	LIN ES	PURPOSE
copilot-assistant	287	AI copilot chat for SOC operators	shift-briefing	122	Daily shift briefing generation
generate-briefing	229	Auto-generate shift briefings	risk-forecast	80	Risk scoring and forecasting
strategic-advisory	185	AI strategic advisory generation	evidence-sign-url	—	Signed URL for evidence vault access
threat-analysis	190	Threat classification and scoring	attendance-anomaly	—	Flag attendance anomalies
patrol-anomaly-detection	176	Detect patrol gaps / missed checkpoints	public-api	54	External client-facing API
incident-ai-summary	122	Summarise incidents with AI	create-user-account	—	Admin user creation
			training-drill-inject	—	Inject training drill scenarios

5 · MODULE STATUS — WHAT'S BUILT VS MISSING

Legend: ✓ **DONE** = frontend + backend complete · ~ **UI DONE** = frontend complete, backend partial · ! **STUB** = page exists, no real data · ✗ **MISSING** = no DB tables yet

MODULE	STATUS	NOTES	MODULE	STATUS	NOTES
Incidents	✓ DONE	1746 lines, useIncidents hook (344 ln), real DB	CCTV	~ UI	205 lines + console (434 ln), needs cctv_cameras.sql
Alarms	~ UI	428 lines, needs Phase 7 migration applied	Body Cam / Evidence	~ UI	58-ln shell + 4 components, needs Phase 6 applied
DOB	~ UI	340 lines, needs dob_soft_delete.sql	Access Control	~ UI	187 lines, needs visitor/armoury migration
Event Security	✓ DONE	1060 lines, substantive implementation	Visitor Access	! STUB	Delegates to component, no DB tables yet
Investigations	✓ DONE	1746 lines, most complete module	Armoury Custody	! STUB	Delegates to component, no DB tables yet
Control Room	~ UI	328 lines, dispatch not fully wired	K9	! STUB	367 lines UI, k9_module.sql not written
MDT Management	~ UI	Pages exist, need real dispatch hooks	CIT	! STUB	632 lines UI, cit_module.sql not written
Patrol Suite	~ UI	72-line shell, sub-pages need hooks	War Room / CMC	! STUB	198 lines UI, cmc_module.sql not written
GPS Patrol Track	! STUB	Page exists, no real GPS hooks yet	Communications	~ UI	73-ln shell, 8 sub-components, mostly mock data
Escort Ops	~ UI	496 lines, needs real hook wiring	Loss Control	~ UI	236 lines, tables exist in schema

MODULE	STATUS	NOTES	MODULE	STATUS	NOTES
Staff Management	~ UI	353 lines, partially reads real DB	Billing & Invoicing	✓ DONE	1220 lines, most complete finance page
HR Recruitment	! STUB	149 lines, UI only	Payroll Runs	! STUB	129 lines UI, needs payroll migration
HR Onboarding	! STUB	135 lines, UI only	Payslips	! STUB	88 lines UI, same migration needed
HR Attendance	! STUB	163 lines, UI only	Expenses	! STUB	139 lines UI, same migration needed
HR Performance	! STUB	131 lines, UI only	Statutory Returns	! STUB	81 lines UI, same migration needed
HR Disciplinary	! STUB	125 lines, UI only	Finance Dashboard	~ UI	390 lines, needs real finance data
Training Mgmt	~ UI	251 lines, tables exist	Client Management	~ UI	327 lines, reads real DB
Training Drills	~ UI	Needs Phase 6 (training_drills table)	Compliance	! STUB	83 lines, UI only

6 · BUILD ORDER — PHASE BY PHASE

PHASE	SECTION	KEY DELIVERABLES	DEPENDS ON
A1	User Management	UserManagement.tsx CRUD, role assignment UI, PendingActivation.tsx	—
A2	Sites & Clients Seed	seed/001_sites.sql, seed/002_staff.sql with real Nairobi sites & staff	A1
A3	Staff Management	useStaff.ts full CRUD, photo upload to storage, search + filter	A2
B1	Alarms	Apply Phase 7 migration, auto-dispatch rules, false alarm log workflow	A2
B2	DOB	Write + apply dob_soft_delete.sql, wire DOB to real DB, auto-create from incidents	A2
B3	Patrol Suite	Write field_portal_phase2.sql, usePatrols hook, GPS tracking, anomaly detection	A2
B4	Communications	useComms.ts for calls/tickets/WhatsApp/SMS, wire all 8 sub-components	A3
B5	Control Room	useDispatch.ts, write hq_connect.sql, wire MDT, directives	A3

PHASE	SECTION	KEY DELIVERABLES	DEPENDS ON
C1	CCTV	Write + apply cctv_cameras.sql, wire CCTVOperatorConsole to real camera registry	A2
C2	Body Cam	Apply Phase 6 migration, create evidence-vault bucket, wire EvidenceLibrary	A2
C3	Armoury + Visitor	Write + apply visitor_access_armoury.sql, wire both modules	A2
C4	K9 Module	Write + apply k9_module.sql (dogs, handlers, deployments, incidents)	A3
C5	CIT	Write + apply cit_module.sql (runs, manifests, vault, routes)	A3
C6	CMC / War Room	Write + apply cmc_module.sql (activations, decisions, resources)	B5
D1	HR Sub-pages	Apply hr_finance_payroll_governance.sql, build useHR.ts, wire 5 HR pages	A3
D2	Finance / Payroll	Same migration as D1, build payroll calc utils, wire pay runs + payslips	D1
E1	Deploy Edge Functions	Set OPENAI_API_KEY, deploy all 14 functions, wire Copilot + Advisory UI	B-D
E2	Executive Dashboards	CEO, HR, Finance, GM, CIT dashboards auto-populate once data exists	E1
E3	RTSI Map (Strategic Advisory)	Mapbox globe, left/right intel panels, NORMAL/CAUTION/CRITICAL colour scheme	E1
F	Governance	Apply Phase 7 audit_log, wire Compliance, Approvals, Directive Log, SOPs	D
G	CS & Courier	Write customer_service_schema.sql + courier migration, wire 9 sub-pages	A3
H	Production Hardening	Multi-tenancy (Phase 7), PWA/offline sync, evidence vault RLS audit, rate limits	F-G

7 · CODING RULES & PATTERNS

Data Query Pattern (Always Use React Query)

```
const { data = [], isLoading } = useQuery({ queryKey: ['incidents', filters], queryFn: async () => { const { data, error } = await supabase.from('incidents').select('*', staff('*')).eq('status', 'open').order('created_at', { ascending: false }); if (error) throw error; return data; }, });
```

Mutation Pattern

```
const create = useMutation({ mutationFn: async (payload: MyType) => { const { data, error } = await supabase.from('my_table').insert(payload).select().single(); if (error) throw error; return data; }, onSuccess: () => queryClient.invalidateQueries({ queryKey: ['my-feature'] }), });
```

Realtime Subscription Pattern

```
useEffect(() => { const channel = supabase.channel('incidents-rt') .on('postgres_changes', { event: '*', schema: 'public', table: 'incidents' }, () => queryClient.invalidateQueries({ queryKey: ['incidents'] })) .subscribe(); return () => { supabase.removeChannel(channel); }, [], []);
```

Migration Rules

```
-- ALWAYS idempotent: CREATE TABLE IF NOT EXISTS public.my_table (...); ALTER TABLE public.my_table ADD COLUMN IF NOT EXISTS new_col TEXT; -- ALWAYS enable RLS: ALTER TABLE public.my_table ENABLE ROW LEVEL SECURITY; -- ALWAYS drop before create: DROP POLICY IF EXISTS "policy_name" ON public.my_table; CREATE POLICY ...
```

CRITICAL DON'TS

- NEVER use React.lazy() directly — always use safeLazy() from @/utils/safeLazy
- NEVER read user roles from sessionStorage — only from useAuth().userRole
- NEVER write a page without a default export — safeLazy requires it
- NEVER modify client.ts or types.ts — auto-generated by Supabase
- NEVER call supabase from inside a component body — always in hooks or query functions
- NEVER access auth.users directly — use public.profiles table
- NEVER hardcode Supabase URLs or keys — always import.meta.env.VITE_*

- NEVER add a route in App.tsx without adding it to WorkspaceShell sidebar nav
- NEVER create a migration without IF NOT EXISTS / DROP POLICY IF EXISTS guards

8 · QUICK REFERENCE			
I NEED TO...	GO TO...	I NEED TO...	GO TO...
Add a new route	src/App.tsx → ConsoleLayout routes	Generate a PDF report	import jsPDF from 'jspdf' + jspdf-autotable
Add a sidebar nav item	src/components/shell/WorkspaceShell.tsx	Show a toast notification	import { toast } from 'sonner'
Check what a user can do	src/config/accessControl.ts	Call an edge function	supabase.functions.invoke('name', { body: {} })
Add a new permission module	accessControl.ts → ModuleKey type + matrix	Know which tables exist	src/integrations/supabase/types.ts
Guard a route by role	in App.tsx	Apply a pending migration	Lovable Cloud → Database → SQL Editor
Guard a UI element		Deploy an edge function	supabase functions deploy
Write to the audit trail	import { logAudit } from '@utils/auditLog'		

Last updated June 2026 — reflects codebase state at the time of analysis. Update this document whenever a major phase is completed.

Prepared by Immersicloud Consulting · immersicloud.org